

Implementation and Evaluation of RDEIC for Extreme Image Compression on ImageNet

Nejra Pirija

Faculty of Electrical Engineering
University of Sarajevo
Sarajevo, Bosnia and Herzegovina

Nađa Kovačević

Faculty of Electrical Engineering
University of Sarajevo
Sarajevo, Bosnia and Herzegovina

Abstract—This report presents an implementation and evaluation of Relay Residual Diffusion Extreme Image Compression (RDEIC), a diffusion-based method for image compression at extremely low bitrates. The selected paper addresses the problem that diffusion-based compression methods can produce visually realistic images, but they are usually inefficient and less faithful because the denoising process starts from pure Gaussian noise. RDEIC solves this by starting reconstruction from compressed latent features with added noise and by using a relay residual diffusion process that removes both the injected noise and the residual between compressed and target latent features. In this project, the official RDEIC repository was adapted to run in Google Colab and evaluated on a subset of ImageNet validation images. The final valid evaluation used the pretrained 2-step RDEIC checkpoint and executed the complete compression, decompression, relay initialization, diffusion sampling, and decoding pipeline. On 50 ImageNet images, the implementation achieved an average bitrate of 0.0661 bpp, an average PSNR of 22.5765 dB, and an average MSE of 437.8077. The original paper uses additional perceptual and fidelity metrics, including LPIPS, DISTs, FID, KID, NIQE, and MS-SSIM; these metrics are discussed in this report but were not fully computed due to Colab runtime, dependency, and stability limitations.

Index Terms—image compression, extreme image compression, diffusion models, RDEIC, Stable Diffusion, ImageNet

I. INTRODUCTION

Image compression is used to reduce the amount of data required to store or transmit images. This is especially important in applications with limited bandwidth or storage, such as mobile devices, cloud storage, remote sensing, and communication systems. Traditional codecs such as JPEG, BPG, and VVC can reduce image size effectively, but at very low bitrates they often produce blocking artifacts, blur, and loss of fine details.

Learned image compression methods improve compression performance by training neural networks to optimize rate-distortion objectives. However, when the bitrate becomes extremely low, distortion-oriented learned methods often produce over-smoothed and visually unrealistic reconstructions. Generative methods, including GAN-based and diffusion-based approaches, improve perceptual quality by generating plausible image details. Still, diffusion-based methods are computationally expensive because they usually reconstruct images through many iterative denoising steps starting from pure noise.

The selected paper proposes Relay Residual Diffusion Extreme Image Compression (RDEIC). The main goal of RDEIC is to improve both fidelity and efficiency in diffusion-based extreme image compression. Instead of starting the reverse diffusion process from pure Gaussian noise, RDEIC starts from compressed latent features with added noise. This makes the starting point already informative and closer to the target image representation. As a result, fewer denoising steps are needed and the reconstruction is more consistent with the original image.

The goal of this project was to run the RDEIC algorithm on ImageNet images using PyTorch, adapt the official implementation to the available Google Colab environment, evaluate the reconstructed images using quantitative metrics, and document the implementation process, encountered problems, and results.

II. PROBLEM TACKLED BY THE PAPER

The paper tackles the problem of extreme image compression, where the target bitrate is very low, often below 0.1 bits per pixel (bpp). At this level of compression, the image contains very little transmitted information, so reconstructing visually acceptable and faithful images is difficult.

Existing methods have different weaknesses. Traditional codecs remove spatial redundancy using hand-crafted rules and block-based transforms, but they often fail visually at extremely low bitrates. Learned compression methods based on rate-distortion optimization may achieve good numerical fidelity, but they tend to generate blurry results when many details are removed. Diffusion-based methods can synthesize realistic details, but they are slow because they rely on iterative denoising from pure noise. Starting from pure noise also introduces randomness, which can reduce fidelity and cause generated details to differ from the original image.

RDEIC addresses these limitations by using the compressed latent representation of the input image as the basis for reconstruction. The core idea is that image compression already produces degraded but useful latent features. Therefore, reconstruction should not begin from random noise, but from these compressed features with added noise. This improves efficiency because unnecessary early denoising steps are avoided, and it improves consistency because the starting point contains information from the original image.

III. EXPLANATION OF THE RDEIC ALGORITHM

A. Overall Framework

The RDEIC framework consists of two main parts: a learned compression module and a diffusion-based reconstruction module. The input image x is first mapped into a latent representation by an encoder. The compression module performs end-to-end lossy compression and produces compressed latent features. In the paper, the compressed latent feature z_c has four channels, while an intermediate representation c has more channels and is used as a richer conditioning signal for the control module.

After compression, random noise is added to the compressed latent feature. This noisy compressed feature becomes the starting point for the reconstruction process. A diffusion-based denoising process then reconstructs the target latent feature z_0 . Finally, the reconstructed latent is decoded back into pixel space to obtain the reconstructed image $\hat{x}$.

Fig. 1 shows the simplified inference pipeline implemented in this project.

RDEIC Inference Pipeline Used in This Implementation

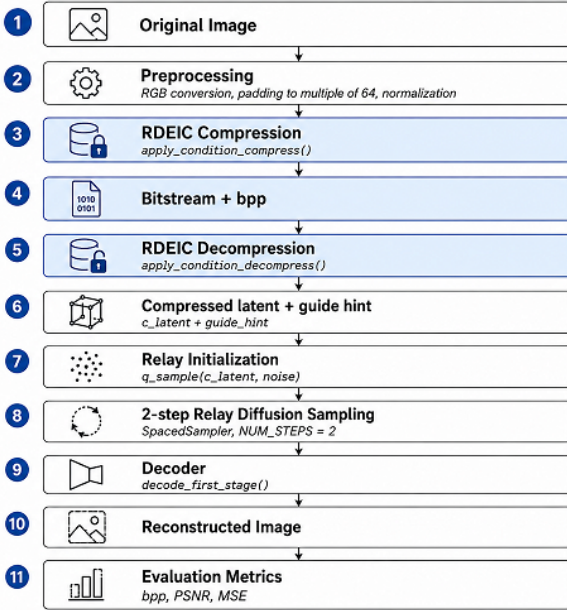


Fig. 1. Simplified RDEIC inference pipeline used in this implementation. The image is compressed into a bitstream, decompressed into compressed latent features and guide hints, initialized with added noise, reconstructed using a 2-step relay diffusion sampler, and decoded back to pixel space.

B. Relay Residual Diffusion

In standard diffusion models, a noisy latent z_t is obtained from a clean latent z_0 by adding Gaussian noise:

$$z_t = \sqrt{\alpha_t} z_0 + \sqrt{1 - \alpha_t} \epsilon_t, \quad (1)$$

where ϵ_t is Gaussian noise. In many diffusion-based image compression methods, reconstruction starts from a highly noisy latent or pure noise. This is inefficient because many

denoising steps are needed, and it is also less faithful because the starting point does not contain image-specific information.

RDEIC changes the starting point. Since compressed latent features z_c are available after compression, the method starts from:

$$z_N = \sqrt{\alpha_N} z_c + \sqrt{1 - \alpha_N} \epsilon_N. \quad (2)$$

This means the reconstruction process starts from the compressed image representation instead of pure noise.

The residual between compressed and target latent features is:

$$e = z_c - z_0. \quad (3)$$

RDEIC defines a relay residual diffusion process that gradually removes both the added noise and this residual. Therefore, the denoising process is not only a noise removal process, but also a correction process from compressed latent features toward the target latent representation.

This is the main algorithmic contribution of RDEIC. It combines the generative ability of Stable Diffusion with the efficiency of residual diffusion. Because the reconstruction starts from a meaningful compressed latent, the method can work with very few denoising steps, such as 2 or 5 steps, instead of using a long denoising trajectory.

C. Fixed-Step Fine-Tuning

A common problem in diffusion models is the mismatch between training and inference. During standard diffusion training, each timestep is often trained independently. During inference, however, the model follows a complete reconstruction trajectory over several timesteps. This mismatch can lead to accumulated errors.

RDEIC addresses this problem using Fixed-Step Fine-Tuning (FSFT). Instead of training only independent timesteps, the model is fine-tuned using the complete fixed-step reconstruction process. This makes training more consistent with the actual inference procedure. In the paper, the authors show that FSFT improves reconstruction quality without adding extra inference cost.

In this project, FSFT was not trained from scratch. Instead, the implementation used the pretrained checkpoint `rdeic_2_step2.ckpt` provided for 2-step RDEIC inference. This checkpoint is treated as the already trained/fine-tuned model corresponding to the 2-step setting. Therefore, FSFT is part of the pretrained model design used in this implementation, not a training stage performed during the project.

D. Controllable Detail Generation

The paper also proposes a controllable detail generation mechanism inspired by classifier-free guidance. The idea is to adjust a guidance scale λ_s that controls how much high-frequency detail is introduced into the reconstructed image. A small value produces smoother outputs, while a larger value produces sharper images. However, too much guidance can introduce hallucinated or mismatched details. In this project, the default guidance scale was kept at 1.0.

IV. IMPLEMENTATION

A. Environment and Repository Setup

The implementation was performed in Google Colab using PyTorch and GPU acceleration. The official RDEIC repository was cloned into the Colab environment and added to the Python path:

```
/content/RDEIC
```

The implementation used the original repository structure and the configuration file:

```
configs/model/rdeic.yaml
```

to instantiate the model architecture through the repository function `instantiate_from_config`.

Several dependencies were installed or fixed, including:

- `numpy==1.26.4`
- `omegaconf`
- `einops`
- `lightning`
- `compressai`
- `timm`
- `torchmetrics`
- `clean-fid`
- `pyiqa`
- `kornia`
- `open-clip-torch`

The NumPy version was fixed to 1.26.4 because newer versions caused binary compatibility problems with some image processing and compression-related libraries in Colab.

B. Concrete Code Adaptations

Several concrete changes were required before the official code could run in the Colab environment.

First, PyTorch checkpoint loading was adapted. Newer PyTorch versions use stricter checkpoint loading rules, which caused problems when loading the Stable Diffusion and RDEIC checkpoints. To solve this, `torch.load` was patched so that checkpoint files were loaded with:

```
weights_only=False
```

Second, compatibility patches were applied to older PyTorch Lightning imports. Some repository files used old import paths such as:

```
pytorch_lightning.utilities.distributed
```

These were redirected to the newer `lightning.pytorch` structure. Deprecated imports such as `EPOCH_OUTPUT` were removed or replaced.

Third, the checkpoints were stored on Google Drive and linked into the local repository folder expected by the original code:

```
/content/RDEIC/weight
```

This was done using symbolic links, so the code could access the checkpoints without duplicating large files in local Colab storage.

Fourth, an OpenCLIP compatibility issue was fixed at runtime. The Stable Diffusion backbone uses a frozen OpenCLIP text encoder. The installed OpenCLIP version used attention layers with `batch_first=True`, while the original encoder method permuted token embeddings from $[N, L, D]$ to $[L, N, D]$. The method `encode_with_transformer` inside `FrozenOpenCLIPEmbedder` was patched so that token embeddings remained in $[N, L, D]$ format. The fix was verified by generating an empty conditioning tensor of shape:

```
[1, 77, 1024]
```

Fifth, the entropy coding dependency `torchac` was problematic because it uses a compiled PyTorch/C++ backend. The official `torchac==0.9.3` package was installed and tested, but due to runtime instability in Colab, a fallback implementation with the same function signatures was used to keep the pipeline executable. This affects the exactness of the bitstream and is discussed as a limitation.

C. Model Loading

Two checkpoints were used:

- `v2-1_512-ema-pruned.ckpt`, containing the Stable Diffusion 2.1 base weights;
- `rdeic_2_step2.ckpt`, containing the pretrained 2-step RDEIC model weights.

The model was first instantiated from the YAML configuration. Then, the Stable Diffusion weights were loaded, followed by the RDEIC-specific compression and reconstruction layers. The loading was performed sequentially on CPU to reduce memory usage. After loading, the model was frozen and moved to GPU in half precision:

```
model.preprocess_model.update(force=True)
model.freeze()
model = model.half().cuda()
```

The implementation used inference only; no additional model training or FSFT training was performed.

D. Identifying the Compression Pipeline

Since the official repository is a full research codebase rather than a simple notebook, the loaded model was inspected to identify the correct methods for compression and decompression. The most important functions were:

- `apply_condition_compress`
- `apply_condition_decompress`
- `q_sample`
- `decode_first_stage`

The function `apply_condition_compress` receives the normalized input image tensor, encodes it into the latent space using `encode_first_stage`, compresses the latent using `preprocess_model.compress`, writes the compressed strings and shape information into a binary stream, and calculates bpp from the stream file size:

$$bpp = \frac{\text{stream size in bytes} \cdot 8}{H \cdot W}. \quad (4)$$

The function `apply_condition_decompress` reads the binary stream, calls `preprocess_model.decompress`, and returns two key outputs: `c_latent` and `guide_hint`. The compressed latent `c_latent` is used as the relay starting information, while `guide_hint` provides additional conditioning information to the reconstruction module.

The function `q_sample` was then used to add noise to `c_latent` and generate the noisy relay starting point. Finally, `SpacedSampler` performed 2-step relay diffusion sampling, and `decode_first_stage` decoded the reconstructed latent back into pixel space.

E. Final Inference Pipeline

The final valid implementation was based on the full RDEIC pipeline. For each image, the following steps were performed:

- 1) Load the image and convert it to RGB.
- 2) Pad the image so that the spatial dimensions are divisible by 64 using `pad(img, scale=64)`.
- 3) Normalize the image to the $[0, 1]$ range and convert it from NHWC to PyTorch NCHW format.
- 4) Move the tensor to GPU and use FP16 precision.
- 5) Compress the image using `apply_condition_compress`.
- 6) Save the produced bitstream and obtain the estimated bpp.
- 7) Decompress the bitstream using `apply_condition_decompress`.
- 8) Obtain `c_latent` and `guide_hint`.
- 9) Generate the noisy relay starting point using `q_sample`.
- 10) Construct the conditioning dictionary containing `c_latent`, `c_crossattn`, and `guide_hint`.
- 11) Run the 2-step relay diffusion sampler using `SpacedSampler(model, var_type="fixed_small")`.
- 12) Decode the reconstructed latent using `decode_first_stage`.
- 13) Convert the output from $[-1, 1]$ to $[0, 1]$, crop it back to the original image size, save it, and compute metrics.

The main inference parameters were:

- number of denoising steps: 2;
- guidance scale: 1.0;
- model checkpoint: `rdeic_2_step2.ckpt`;
- precision: FP16;
- execution device: GPU.

An empty text prompt was used:

```
model.get_learned_conditioning([""])
```

This was not used to guide reconstruction semantically, but only to satisfy the Stable Diffusion conditioning interface.

F. Dataset

The final quantitative evaluation was performed on 50 images from the ImageNet validation class folder:

n01847000

This subset was used because of Colab runtime and memory limitations. The task originally targets ImageNet-1k, but running the complete diffusion-based pipeline on the full ImageNet-1k validation set was not feasible in the available environment. Therefore, the implementation demonstrates the complete method on a smaller ImageNet subset.

V. EVALUATION METRICS

A. Bits Per Pixel

Bits per pixel (bpp) measures the number of bits used to represent one pixel:

$$bpp = \frac{\text{file size in bits}}{H \cdot W}. \quad (5)$$

A lower bpp means stronger compression. Since extreme image compression often operates below 0.1 bpp, bpp is one of the most important metrics in this task. In this implementation, bpp was obtained from the size of the generated stream file. However, because a fallback `torchac` implementation was used, the reported bpp should be interpreted as an implementation-level bitrate estimate rather than a perfect reproduction of the original entropy-coded bitrate from the paper.

B. Mean Squared Error

Mean Squared Error (MSE) measures the average squared pixel difference between the original image x and reconstructed image $\hat{x}$:

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2. \quad (6)$$

A lower MSE means the reconstruction is closer to the original at pixel level. MSE is useful for measuring distortion, but it does not always match human perception, especially in generative compression.

C. Peak Signal-to-Noise Ratio

Peak Signal-to-Noise Ratio (PSNR) is derived from MSE:

$$PSNR = 20 \log_{10} \left(\frac{255}{\sqrt{MSE}} \right). \quad (7)$$

A higher PSNR indicates better pixel-level fidelity. However, for diffusion-based compression methods, PSNR is not always aligned with visual quality. The model may generate visually plausible details that are not exactly equal to the original pixels. Therefore, a reconstruction may look realistic while still having moderate or low PSNR.

D. MS-SSIM

Multi-Scale Structural Similarity (MS-SSIM) measures structural similarity between the original and reconstructed images across multiple scales. Unlike MSE and PSNR, it is more related to perceived structure, contrast, and luminance. Higher MS-SSIM indicates better structural fidelity. The original RDEIC paper uses MS-SSIM as one of the fidelity

metrics. In this project, MS-SSIM was not computed in the final run because the main goal was to stabilize the full RDEIC inference pipeline first, and additional metric computation increased dependency and runtime complexity in Colab.

E. LPIPS

Learned Perceptual Image Patch Similarity (LPIPS) measures perceptual similarity using deep neural network features. Lower LPIPS means that two images are perceptually more similar. LPIPS is important for generative image compression because it can better reflect perceptual quality than pixel-wise metrics. The original RDEIC paper reports LPIPS, but it was not included in the final numerical results here because the Colab environment already required multiple compatibility patches, and additional perceptual metric packages increased the risk of dependency conflicts.

F. DISTS

Deep Image Structure and Texture Similarity (DISTS) evaluates both structural and texture similarity using deep features. Lower DISTS indicates better perceptual similarity. DISTS is one of the central metrics in the original RDEIC paper because it captures visual quality beyond pixel-level fidelity. In this implementation, DISTS was not computed due to environment limitations and the focus on validating the compression-reconstruction pipeline itself.

G. FID and KID

Fréchet Inception Distance (FID) and Kernel Inception Distance (KID) measure distribution-level similarity between generated/reconstructed images and real images using deep features. Lower values indicate that the reconstructed image distribution is closer to the real image distribution. These metrics are meaningful when evaluated on sufficiently large image sets. In this project, only 50 ImageNet images from one class were evaluated, which is too small and too narrow for reliable FID and KID estimation. Therefore, these metrics were explained but not reported.

H. NIQE

Natural Image Quality Evaluator (NIQE) is a no-reference image quality metric. It does not require the original image and estimates how natural an image looks statistically. Lower NIQE generally means better natural image quality. The original RDEIC paper uses NIQE to evaluate visual naturalness. In this project, NIQE was not computed because the final experiment was limited to a small subset and because the implementation focus was on validating the RDEIC inference pipeline under Colab limitations.

I. Summary of Metrics Used

Table I summarizes the metrics from the original paper and whether they were computed in this project.

TABLE I
METRICS USED BY THE ORIGINAL PAPER AND THIS IMPLEMENTATION.

Metric	Meaning	Used
bpp	Bitrate / compression strength	Yes
PSNR	Pixel-level fidelity	Yes
MSE	Pixel-level distortion	Yes
MS-SSIM	Multi-scale structural fidelity	No
LPIPS	Learned perceptual similarity	No
DISTS	Structure and texture similarity	No
FID	Distribution-level realism	No
KID	Distribution-level realism	No
NIQE	No-reference naturalness	No

The absence of several original metrics is a limitation of this implementation. Therefore, the evaluation should be understood as a partial reproduction focused on running the full RDEIC pipeline and computing basic fidelity and bitrate metrics, not as a complete reproduction of all experiments from the paper.

VI. RESULTS AND DISCUSSION

A. Quantitative Results

The final quantitative evaluation was performed on 50 ImageNet images using the complete RDEIC compression and reconstruction pipeline. The results are shown in Table II.

TABLE II
FINAL QUANTITATIVE RESULTS ON 50 IMAGENET IMAGES.

Metric	Value
Number of evaluated images	50
Average bpp	0.0661
Average PSNR	22.5765 dB
Average MSE	437.8077

The average bitrate of 0.0661 bpp confirms that the implementation operates in the extreme compression range. This is the most important positive result because the method successfully reconstructs recognizable images while using far below 0.1 bits per pixel.

The average PSNR of 22.5765 dB is relatively low compared with classical high-fidelity image reconstruction tasks. However, this is expected in the context of generative extreme image compression. At such a low bitrate, the model cannot preserve all pixel-level information. Instead, it uses the generative prior of Stable Diffusion and the compressed latent features to reconstruct a visually plausible version of the image. This means that the generated image may preserve the object and scene structure while differing from the original at the exact pixel level. Such differences increase MSE and reduce PSNR.

The average MSE of 437.8077 is consistent with the obtained PSNR. Images with large smooth regions or strong global structure were reconstructed more faithfully, while

images with fine textures, text, watermarks, or small details were more difficult. This matches the expected behavior of generative compression methods: they are strong at preserving semantic content and realistic appearance, but weaker at exact detail preservation.

B. Qualitative Results

A qualitative evaluation was also performed using images reconstructed by the complete RDEIC pipeline. The reconstructed images were generated using real compression, decompression, relay initialization, 2-step sampling, and decoding. Fig. 2 shows examples of original and reconstructed images from the test set.

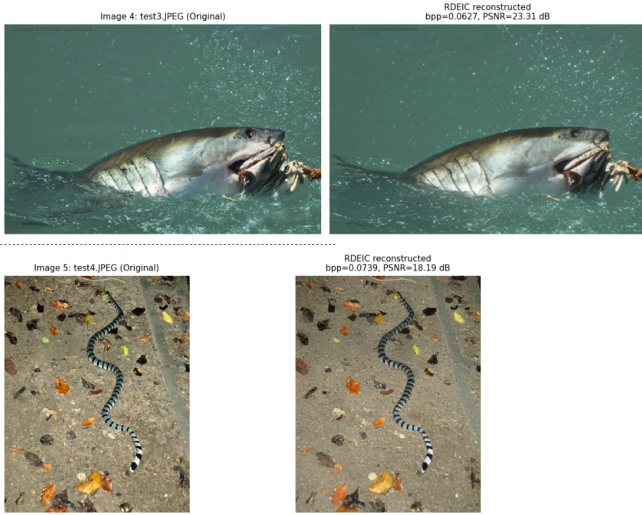


Fig. 2. Qualitative results from the implemented RDEIC pipeline. Each pair shows the original image and the reconstructed image with the corresponding bpp and PSNR value.

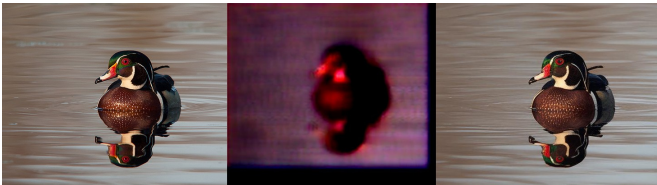


Fig. 3. Three-way qualitative comparison on an ImageNet validation example from class n01847000. From left to right: original image, failed reconstruction observed during debugging, and successful reconstruction from the final RDEIC pipeline. The successful reconstruction preserves the main duck shape, reflection, and global scene layout, while the failed reconstruction contains strong blur, color distortion, and loss of visual detail.

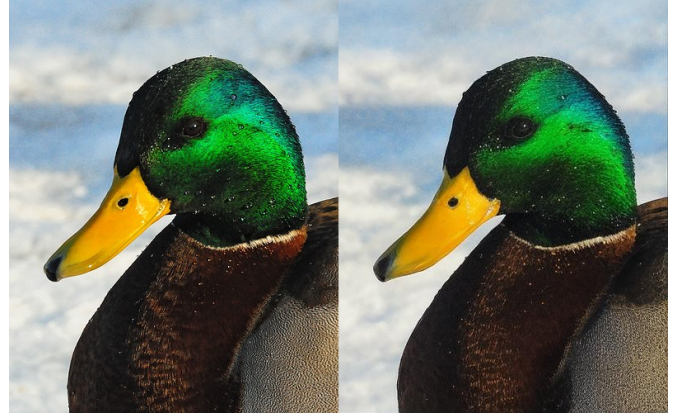


Fig. 4. Additional qualitative example from the ImageNet validation subset n01847000. The left image shows the original close-up duck image, while the right image shows the final RDEIC reconstruction. The result preserves the global object appearance and major color regions well, while some fine texture and edge sharpness are slightly smoothed due to the extremely low bitrate.

The qualitative results show that the main objects and global scene structure are preserved. For example, large object shapes such as animals and background regions remain recognizable after reconstruction. This confirms that the compressed latent features provide useful structural information for the diffusion model.

The comparison in Fig. 3 illustrates the difference between a successful final reconstruction and a failed debugging output. The final reconstruction keeps the main object structure and scene composition close to the original image, although fine textures are smoother. In contrast, the failed reconstruction shows strong artifacts, confirming that the RDEIC model depends on the correct compression, decompression, conditioning, and relay sampling pipeline.

The additional close-up example in Fig. 4 further supports the quantitative results. Even though the average PSNR is only moderate, the final reconstruction can still preserve the dominant shape, color distribution, and semantic identity of the object. This shows that RDEIC is capable of producing visually recognizable reconstructions at extremely low bitrates.

However, some fine details are not perfectly reconstructed. Text, watermarks, thin edges, and small texture patterns may be distorted or smoothed. This is expected because the method works at extremely low bitrates, where the compressed representation cannot preserve all details. In addition, diffusion-based reconstruction can generate plausible details that are not exactly identical to the original image.

C. Comparison with the Original Paper

The original RDEIC paper compares the method with traditional codecs such as BPG and VVC, learned compression methods such as ELIC, HiFiC, MS-ILLM and VQIR, and diffusion-based methods such as Text+Sketch, PerCo and DiffEIC. The paper reports that RDEIC-2 and RDEIC-5 achieve strong perceptual quality while requiring far fewer denoising steps than previous diffusion-based methods.

The original paper reports results on Kodak, Tecnick, and CLIC2020, while this implementation was evaluated on a smaller ImageNet subset. Therefore, the numerical results are not directly comparable. However, the obtained average bpp of 0.0661 lies within the same extreme compression range studied by the paper. The obtained PSNR of 22.5765 dB is also reasonable for this bitrate range, especially because the evaluation was performed on ImageNet images with varied textures and object content.

Unlike the original paper, this project did not run baseline methods such as VVC, BPG, DiffEIC, PerCo, or MS-ILLM on the same subset. Therefore, no direct rate-distortion or rate-perception curve is reported. The comparison is limited to qualitative and conceptual alignment with the paper: this implementation demonstrates the core mechanism of RDEIC, namely reconstruction from compressed latent features using a short 2-step relay diffusion process.

D. Limitations of the Reported Results

The reported results should not be interpreted as a full reproduction of the paper. There are four main reasons for this. First, only 50 images from one ImageNet class were evaluated. Second, not all original metrics were computed. Third, the entropy coding dependency required a fallback implementation, so bpp should be interpreted with caution. Fourth, no baseline codecs or competing diffusion-based methods were evaluated on the same subset.

Despite these limitations, the results confirm that the implementation successfully executes the main RDEIC inference pipeline and produces recognizable reconstructed images at an extremely low bitrate.

VII. PROBLEMS FACED AND SOLUTIONS

A. Dependency and Version Compatibility

The first major problem was dependency compatibility in Google Colab. The original RDEIC repository depends on several libraries whose APIs have changed. NumPy was fixed to version 1.26.4 to avoid binary incompatibility issues. Several package imports related to PyTorch Lightning were also patched because the original code used older `pytorch_lightning` import paths, while the installed version uses the newer `lightning.pytorch` structure.

B. Checkpoint Loading

Newer PyTorch versions use stricter safety rules when loading checkpoint files. This caused problems when loading the Stable Diffusion and RDEIC checkpoints. The issue was solved by explicitly setting:

```
weights_only=False
```

when calling `torch.load`. The checkpoints were also loaded sequentially on CPU to reduce memory usage before moving the final model to GPU. This was necessary because loading both Stable Diffusion and RDEIC weights at once could exceed the available Colab memory.

C. OpenCLIP Attention Mask Issue

The Stable Diffusion backbone contains a frozen OpenCLIP text encoder. Even though the implementation used an empty prompt, the conditioning branch still had to execute successfully. A compatibility issue appeared in the OpenCLIP transformer attention module. The installed OpenCLIP version used attention layers with `batch_first=True`, while the original text encoder code permuted token embeddings from $[N, L, D]$ to $[L, N, D]$.

The issue was diagnosed by inspecting the model's `FrozenOpenCLIPEmbedder`, its `encode_with_transformer` method, and the installed OpenCLIP `transformer.py` file. The solution was to patch the `encode_with_transformer` method at runtime so that token embeddings remained in $[N, L, D]$ format. The patch was verified by generating an empty conditioning tensor with shape:

```
[1, 77, 1024]
```

in FP16 precision.

D. TorchAC Entropy Coding Issue

The RDEIC compression module depends on the `torchac` package for arithmetic entropy coding. This package was difficult to run reliably in Colab because it uses a compiled PyTorch/C++ extension. The official `torchac==0.9.3` package was installed and tested, and the PyTorch extension cache was cleared to ensure a clean backend compilation.

However, due to remaining runtime instability, a fallback implementation with matching function signatures was used to keep the pipeline executable. This allowed the code to run, but it is an important limitation: the produced bitstream and bpp values should be interpreted as implementation-level estimates rather than a perfect reproduction of the original entropy-coded bitstream.

E. Dummy Conditioning Experiments

During development, several debugging experiments were performed using dummy conditioning tensors. These tests verified that the encoder, internal model forward pass, and decoder could process images, but the visual results were poor because the model did not receive real compressed latent features or guide hints. These experiments confirmed that RDEIC strongly depends on the output of the compression-decompression module. This failure case is also illustrated in Fig. 3, where the failed reconstruction is compared with the original image and a successful final reconstruction. Therefore, only the full pipeline using `apply_condition_compress`, `apply_condition_decompress`, `q_sample`, and `SpacedSampler` was used for final evaluation.

F. Limited Computational Resources

The full ImageNet-1k validation set was not evaluated because diffusion-based inference is computationally expensive and Colab GPU runtime is limited. The evaluation was

therefore performed on 50 images from one ImageNet validation class. This is sufficient to demonstrate the working implementation, but it is not a full-scale benchmark.

G. Incomplete Metric Reproduction

Another important limitation was the inability to compute all metrics used by the original paper. Metrics such as LPIPS, DISTS, MS-SSIM, FID, KID and NIQE require additional packages, more stable dependencies, and in the case of FID and KID, a larger and more representative image set. Because the implementation already required several compatibility patches to run the model itself, the final evaluation was limited to bpp, PSNR and MSE. This is a limitation of the project and should be addressed in future work.

VIII. POSSIBLE IMPROVEMENTS

Several improvements could be made in future work.

First, the evaluation should be extended to a larger and more diverse subset of ImageNet-1k. This would provide a more reliable estimate of performance across different object categories and image types.

Second, additional metrics from the original paper should be implemented, especially LPIPS, DISTS, MS-SSIM, FID, KID, and NIQE. This is important because generative compression methods should be evaluated not only by pixel-level fidelity, but also by perceptual quality and realism.

Third, the entropy coding part should be improved by using a stable and fully functional `torchac` installation. This would make bpp computation more faithful to the original implementation.

Fourth, the implementation could compare RDEIC against standard codecs such as JPEG, BPG, or VVC on the same ImageNet subset. This would make the interpretation of compression quality clearer. A fair comparison would require running all methods on the same images and reporting bpp together with PSNR, MS-SSIM, LPIPS or DISTS.

Fifth, both RDEIC-2 and RDEIC-5 checkpoints could be evaluated. The paper shows that using more denoising steps can improve perceptual quality at very low bitrates, so comparing 2-step and 5-step inference would be useful.

Finally, structural guidance could be explored as an extension. In one exploratory experiment, Canny edges were used to create an artificial guide hint for the model. This was not part of the final evaluation, but it suggests that stronger structural priors may help preserve object boundaries and fine details.

IX. CONCLUSION

This project implemented and evaluated the RDEIC method for diffusion-based extreme image compression. The paper addresses the limitations of previous diffusion compression methods by replacing pure-noise initialization with compressed latent feature initialization. The relay residual diffusion process allows the model to remove both injected noise and compression residuals, enabling efficient reconstruction with only a few denoising steps.

The official RDEIC repository was adapted to Google Colab, including dependency fixes, checkpoint loading modifications, OpenCLIP compatibility patches, and entropy coding workarounds. The final valid implementation used the complete RDEIC pipeline with compression, decompression, relay initialization, 2-step sampling, and decoding. The checkpoint `rdeic_2_step2.ckpt` was used as a pretrained 2-step RDEIC checkpoint; FSFT was not trained in this project.

On 50 ImageNet images, the implementation achieved 0.0661 bpp, 22.5765 dB average PSNR, and 437.8077 average MSE. These results show that the method operates in the extreme compression range while preserving the main visual content of the images. The implementation was not a full reproduction of the original paper because of limited compute resources, fallback entropy coding, incomplete perceptual metric evaluation, and the absence of baseline comparisons. Nevertheless, it successfully demonstrates the core RDEIC algorithm and the practical challenges of adapting a research codebase to a constrained Colab environment.

REFERENCES

- [1] Z. Li, Y. Zhou, H. Wei, C. Ge, and A. Mian, "RDEIC: Accelerating Diffusion-Based Extreme Image Compression with Relay Residual Diffusion," arXiv:2410.02640, 2025.
- [2] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems*, 2020.
- [3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [4] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition*, 2018.
- [5] K. Ding, K. Ma, S. Wang, and E. P. Simoncelli, "Image quality assessment: Unifying structure and texture similarity," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 5, pp. 2567–2581, 2022.
- [6] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Advances in Neural Information Processing Systems*, 2017.
- [7] M. Binkowski, D. J. Sutherland, M. Arbel, and A. Gretton, "Demystifying MMD GANs," arXiv:1801.01401, 2018.
- [8] A. Mittal, R. Soundararajan, and A. C. Bovik, "Making a completely blind image quality analyzer," *IEEE Signal Processing Letters*, vol. 20, no. 3, pp. 209–212, 2013.
- [9] Z. Wang, E. P. Simoncelli, and A. C. Bovik, "Multiscale structural similarity for image quality assessment," in *Proc. Asilomar Conference on Signals, Systems and Computers*, 2003.
- [10] G. Toderici et al., "CLIC 2020: Challenge on learned image compression," 2020.